

LectureDoc² Tutorial



Author: *Prof. Dr. Michael Eichberg*

Version: 2.1

Introduction

LectureDoc is an authoring system for creating lecture material; i. e., lecture slides, notes and exercises from a single document.

A single LectureDoc document contains discussions of topics which are then used as templates for creating advanced slides as well as a standard document. LectureDoc is intended to be used in combination with rst2ld (reStructuredText to LectureDoc) which is a tool that converts reStructuredText documents into LectureDoc and makes authoring slides as easy as writing a text document.

This tutorial is written in reStructuredText (*rst* in the following) and can be used as a template for creating your own lecture slides. The *code* of this tutorial is available on GitHub: https://github.com/Delors/reStructuredTextToLectureDoc2/blob/main/ld_base_example.en.rst.

Basics

A basic slide consists of a (section) header and some reStructuredText content.

Example

Basics

A basic slide consists of a (section) header
and some reStructuredText content.

Embedding Formulae

Embed math equations using reStructuredText's default directive (`.. math::`) and role (`:math:`...``).

Example

The following rst fragment:

```
1 Computation in :math:`GF(2)`:
2
3 .. math::
4
5     \begin{matrix}
6         1 + 1 & = & 1 - 1 & = & 0 \\
7         1 + 0 & = & 1 - 0 & = & 1 \\
8         0 + 1 & = & 0 - 1 & = & 1 \\
9     \end{matrix}
```

Will render like this:

Computation in $GF(2)$:

$1 + 1$	$=$	$1 - 1$	$=$	0
$1 + 0$	$=$	$1 - 0$	$=$	1
$0 + 1$	$=$	$0 - 1$	$=$	1

You can use `no-title` in combination with the `class` directive to avoid that the title is shown on the slide/document. However, the title is still used for indexes.

Example

```
1 .. class:: no-title
2
3 I will only show up in an index...
4 -----
```

Animation

Basic *appear* animations can be created using the (CSS) class `incremental`^[1]. You can also define a corresponding custom role (`.. role:: incremental`) to animate parts of a text.

Example

```
1 Animation
2 -----
3
4 Basic *appear* animations can be created using the (CSS) class
5 ``incremental``. You can also define a corresponding custom role
6 (``.. role:: incremental``) :incremental: to animate parts of a text.
7
8 .. example::
9     :class: incremental
10
11     ...
```

[1] Animation progress can be reset by pressing the `r` key.

Animation of Lists

In case of (un-)ordered and definition lists (`ol` or `ul` in HTML) it is sufficient to associate the class `incremental-list` using the `class` directive with the list. It is also possible, to only specify the `incremental` class attribute for the required list items.

 Example

The following code:

```
1 .. class:: incremental-list
2
3 - this
4 - is
5 - a test
```

Will render incrementally like this:

- this
- is
- a test

Slide Dimensions

The slide dimensions can be controlled by specifying the corresponding meta information. If not specified, the dimension is set to **1920 × 1200** (default); i.e., a ratio of 16:10.

Example

In HTML documents add the following meta tag:

```
<meta name="slide-dimensions" content="1600x1200">
```

In reStructuredText documents add at the beginning:

```
.. meta::  
   :slide-dimensions: 1600x1200
```


Associating a document with a unique id

Many functions in LectureDoc2 - e.g. persistence of the slide progress - require that a document is associated with a unique id. This id can be set using the meta directive. If no id is set, the respective functions are not available.

Example

```
1 .. meta::  
2   :id: lecturedoc2-tutorial  
3   :description: LectureDoc2 Tutorial  
4   :author: Michael Eichberg  
5   :license: Released under the terms of the `2-Clause BSD license`.
```

Adding Supplemental Information

Adding information that should not be on the slides, but provide additional information/explanations, can be added using the `supplemental` directive.

Example

```
1 .. supplemental::  
2  
3     **Formatting Slides**  
4  
5     Formatting slides is done using classes and roles.
```

Formatting Slides

Creating heavily formatted slides is easily possible using rst directives and roles which are mapped to CSS classes.

1. Structuring Documents

Creating Sections

Creating a slide which marks the beginning of a new section can be done using the `new-section` class.

Example

```
.. class:: new-section
```

Structuring Documents

```
.. class:: new-subsection
```

Creating Sections

Slide Transitions

Slide transitions can be controlled using the `transition-...` classes^[2]:

- `transition-fade`
- `transition-move-left`
- `transition-move-to-top`
- `transition-scale`
- `transition-flip`

Example

```
1 .. class:: transition-move-to-top
2
3 Slide Transitions
4 -----
```

^[2] See the LectureDoc2 Cheat Sheet for a comprehensive list of predefined transitions.

Adding Code

Adding code can be done using reStructuredText's code directive.

Example

The following code:

```
1 .. code:: python
2     :number-lines:
3
4     for i in range(0,10):
5         print(i)
```

Will render like this:

```
1 for i in range(0,10):
2 print(i)
```

Links to External Resources

LectureDoc2 supports links to external resources:



Definition

- <https://github.com/Delors/LectureDoc2>
- LectureDoc2 Sourcecode



Example

```
1 LectureDoc2 supports links to external resources:
2
3 - https://github.com/Delors/LectureDoc2
4 - `LectureDoc2 Sourcecode <https://github.com/Delors/LectureDoc2>`_
```

Links to Internal Targets

LectureDoc2 supports links to external resources:

- The title of a slide can be used as a link target ➡ **Advanced Formatting**
- An element which is explicitly marked as a target can be used as a link target:
 - ➡ **Link Target in Incremental Block**

Example

Slide with explicit marked-up element:

```
1 Adv. Formatting
2 -----
3
4 .. container:: incremental
5
6 .. Link Target in Block:
7
8 See the LectureDoc2 Cheat Sheet.
```

References are defined as follows:

```
1 Links to internal targets:
2
3 - Link to slide: `Adv. Formatting`_
4 - Link to a marked-up element:
5
6 `Link Target in Block`_
```


Scientific Citations

Citations are fully supported in LectureDoc2.

A reference to a book: [\[Martin2017\]](#) (Details are found in the bibliography (see next slide)).

Example

A reference to a book: [\[Martin2017\]](#)_

Bibliography

- [\[Martin2017\]](#) Clean Architecture: A Craftsman's Guide to Software Structure and Design; Robert C. Martin, Addison-Wesley, 2017

■ ...

Example

.. [\[Martin2017\]](#) Clean Architecture: ...; Robert C. Martin, Addison-Wesley, 2017

Advanced Formatting

LectureDoc comes with a set of predefined (CSS) classes that can be used to format the slides. Some of these classes have explicit support by LectureDoc and will be rendered differently in the different situations (e.g., document view vs. slide view will render *stacked layouts* or *supplemental information* differently).

■ red

■ peripheral

■ ~~obsolete~~

See the LectureDoc2 Cheat Sheet for a comprehensive list of predefined CSS classes.

Stacked layouts

Stacked layouts enables updating parts of a slide by putting the content into layers and then showing the layers incrementally.

Example

~~XXXXXXXXXXXXXXXXXXXXX.~~

```
1 .. deck:: monospaced
2
3 .. card::
4
5   :gray: This text is gray.
6
7 .. card:: overlay
8
9   XXXXXXXXXXXXXXXXXXXX
```

Presenter-Notes

Presenter notes can be added to a slide using the `presenter-note` directive.

A presenter note - including its presence - is only visible after entering the master password (press `m` and then enter: `123456`).

Example

```
1 .. presenter-note::  
2  
3     This is a presenter note.  
4  
5     It is only visible after entering the master password (123456).
```

Integrated Exercises

Exercises can be integrated into the slide set.

Example

1.1. Exercise: 1+1

Compute: $\sqrt{2} = ?$

To unlock the solution go to the document view (press `c`) and enter the password (sqrt).

```
1 .. exercise:: Exercise: 1+1
2
3   Compute: :math: '\sqrt{2} = ?'.
4
5   .. solution::
6     :pwd: sqrt
7
8     Solution: :math: '1,4142135624'.
```

If you have multiple exercises, you can define a master password (123456) to unlock all solutions at once (press `m` to open the dialog).

```
.. meta::
  :master-password: 123456
```

2. Images

Example

```
1 .. class:: padding-none no-title transition-scale
2
3 Image in the Background
4 -----
5
6 .. deck::
7
8 .. card::
9
10 .. image:: ld_base_example/tag_cloud.webp
11      :width: 100%
12      :align: center
13
14 .. card:: overlay
15
16 Content on the slide...
```


Inline SVGs

Inline SVGs are fully supported by LectureDoc, but styles and definitions that are used in multiple inline SVGs have to be centralized!

This is due to the copying of the slide templates which - if you use ids to reference definitions in the SVGs - makes them no longer unique. This is a violation of the spec and causes troubles in Chrome and Firefox.

Adding Shared Definitions

To add shared SVG definitions, use the `.. meta::` directive and the `:svg-defs:` property.

Example

```
1 :svg-defs:
2   <marker id="arrow"
3     viewBox="0 0 10 10" refX="10" refY="5"
4     markerUnits="strokeWidth"
5     markerWidth="4" markerHeight="4"
6     orient="auto-start-reverse">
7     <path d="M 0 0 L 10 5 L 0 10 z" /></marker>
8   <g id="star">
9     <polygon
10      class="star"
11      points="12,2 15,9 22,9 16,14 18,21 12,17 6,21 8,14 2,9 9,9"
12      fill="gold"
13      style="transform: scale(0.05)"/></g>
```

Defining Shared Styles

To add shared SVG styles, use the `.. meta::` directive and the `:svg-style:` property.

Example

```
1 :svg-style:
2   .std-line {
3     stroke:rgb(0,0,0);
4     stroke-width:0.2ch;
5     stroke-dasharray: 1 1;
6   }
```

Example

Use of the previously defined class `std-line` (line 7), `star` (`href="#star"` line 8) and `arrow` (`marker-end="url(#arrow)"` line 11).

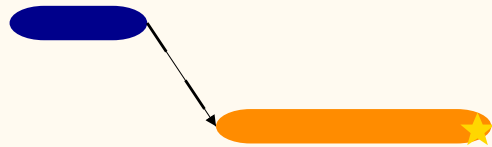
```
1 <div style="width: 90ch; height:16ch">
2 <svg version="1.1" xmlns="http://www.w3.org/2000/svg"
3     viewBox="0 0 48 8" font-size="0.75" >
4   <rect width="4" height="1" x="8" y="3" rx="1" ry="1"
5     style="fill:darkblue" />
6   <line x1="4" y1="3.5" x2="8" y2="3.5"
7     class="std-line"
8     marker-end="url(#arrow)"/>
9   <rect width="8" height="1" x="14" y="6" rx="1" ry="1"
```

```
10 | style="fill:darkorange" />  
11 | <use href="#star" x="21" y="6" />  
12 | </svg>  
13 | </div>
```

The example also demonstrates how to define an SVG whose size is completely dependent on the size of the surrounding font-size.

Example

Rendered SVG



Embedding Images

In general, embeddings images is done using the image directive. However, due to the fact that we render the topic once as classical slides and once in a document-oriented way, it is important to understand how LectureDoc handles images.

General Guidelines

In general an image should be designed/generated/created with its usage on a slide in mind. That is, an image should fit on a slide with a *logical resolution* of 1900 by 1080/1200 pixels. Hence, if text is found on the image it should not be smaller than 30px; ideally it should use the same font size as used by the slide. Those images should then be added to the document using the image directive. In this case it is optional to specify the width and/or height. Such images will be automatically scaled by LectureDoc when the content is shown in the document view. The scaling factor is determined by the ratio between the default font-size used for the document and the default font-size used for the slides.

HighDPI Images

As said, images are generally assumed to have a resolution that fits a slide. However, in many cases the source image may have a resolution that is (much) higher. In this case, it is possible to scale the image using the directive's width and/or height attribute. LectureDoc will then update the width and height attributes when shown in document mode. This requires that the images' width and heights are given in pixels.

Images/SVGs With Font-size Dependent Sizing

SVGs where the size is (alread) dependent on the font-size should not specify any width or height attributes.